

KRYPTOGRAFIE

V předchozí kapitole o hardwaru jsme sledovali, jak počítač ukládá a zpracovává bity. Samotná bitová reprezentace však neříká nic o tom, kdo smí uložená data přečíst, zda je někdo cestou změnil ani od koho skutečně pocházejí. Jakmile zprávu posíláme přes síť nebo ji ukládáme na zařízení, ke kterému mohou získat přístup další lidé, potřebujeme tyto otázky řešit výpočetními prostředky.

Právě zde nastupuje kryptografie. Nejprve si ujasníme její cíle a doplníme základy pravděpodobnosti, bez nichž nelze moderní pojem bezpečnosti přesně formulovat. Poté začneme historickými substitučními šiframi a přejdeme k symetrickému a asymetrickému šifrování. Na algoritmu RSA uvidíme, jak lze bezpečnost spojit s obtížností matematického problému. Závěr kapitoly bude patřit pseudonáhodným generátorům, jednosměrným a hešovacím funkcím, standardu SHA a elektronickému podpisu.

1 Co je to kryptografie?

Název *kryptologie* vznikl spojením řeckých slov označujících něco skrytého a nauku. Kryptologie je zastřešující obor zabývající se ochranou informací i překonáváním této ochrany. *Kryptografie* zkoumá návrh a vlastnosti metod, které mají informace chránit, zatímco *kryptanalýza* hledá způsoby, jak takové metody prolomit. Tyto dvě oblasti se navzájem doplňují: šifru nelze považovat za bezpečnou jen proto, že její autor zatím žádný útok nenašel.

Základní situaci budeme popisovat pomocí tradičních postav Alice a Boba. Alice chce Bobovi poslat zprávu

$$m \in \{0, 1\}^n.$$

Komunikační kanál však sleduje záškodník. Ten může přenášená data odposlechnout, pozměnit, zadržet nebo nahradit vlastní zprávou. Předpokládáme přitom, že záškodník zná používané algoritmy. Tajemstvím nemá být způsob šifrování, ale pouze vhodně zvolený klíč.

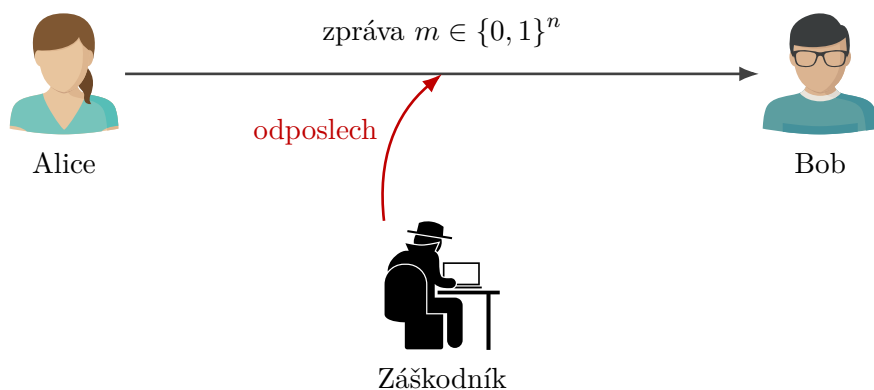


Figure 1: Alice posílá Bobovi zprávu přes kanál sledovaný záškodníkem.

Kryptografie neřeší pouze utajení obsahu. V praxi se obvykle snažíme dosáhnout několika odlišných cílů:

- **důvěrnost** – obsah zprávy má být čitelný pouze oprávněným příjemcům;
- **integrita** – příjemce má poznat, zda byla zpráva změněna;
- **autenticita** – příjemce má umět ověřit, od koho zpráva pochází;

- **doložitelnost původu** – odesílatel nemá moci snadno popřít, že zprávu vytvořil; přesný právní význam však závisí na použitém typu podpisu a okolnostech.

Šifrování se zaměřuje především na důvěrnost, samo o sobě však automaticky nezaručuje integritu ani autenticitu. Hešovací funkce pomáhají odhalit změnu dat, ale neříkají, kdo otisk vytvořil. Elektronický podpis propojí obsah zprávy se soukromým klíčem podepisující osoby. Jednotlivé nástroje proto budeme posuzovat podle toho, jaký bezpečnostní problém skutečně řeší.

Bezpečnost moderního kryptografického systému nemá stát na utajení algoritmu. Algoritmus může být veřejný a podrobně zkoumaný; tajný zůstává klíč. Díky tomu lze systém analyzovat a případné slabiny odhalit dříve, než budou zneužity.

1.1 Šifrování a dešifrování

Zprávu před zašifrováním nazýváme *otevřený text*. Šifrovací algoritmus **E** ji spolu s klíčem převede na *šifrový text* c . Dešifrovací algoritmus **D** má oprávněnému příjemci umožnit získat původní zprávu:

$$c = \mathbf{E}(m, k), \quad m = \mathbf{D}(c, k).$$

Podle toho, zda se pro obě operace používá tentýž tajný klíč, nebo dvojice veřejného a soukromého klíče, budeme později rozlišovat symetrickou a asymetrickou kryptografii.

Požadavek na správné dešifrování nazýváme *korektnost*. V nejjednodušším symetrickém případě má pro každou dovolenou zprávu m a klíč k platit

$$\mathbf{D}(\mathbf{E}(m, k), k) = m.$$

Korektnost ale ještě neznamená bezpečnost. I velmi snadno prolomitelná historická šifra může být korektní, pokud Bob se správným klíčem vždy získá původní zprávu.

V této kapitole pracujeme převážně s bitovými řetězci. Text, obrázek i jiný soubor lze před šifrováním převést na posloupnost bitů, takže model $m \in \{0, 1\}^n$ není omezen pouze na krátké textové zprávy.

2 Odbočka k pravděpodobnosti

V historických šifrách se často ptáme, kolik klíčů musí záškodník vyzkoušet. Moderní kryptografie však používá také náhodně generované klíče a algoritmy, které při běhu provádějí náhodné volby. Bezpečnost potom nepopisujeme pouze větou „útok je těžký“, ale pravděpodobností, s jakou může uspět. Proto si nejprve vyložíme základní pojmy potřebné v dalších sekcích.

2.1 Náhodný pokus, výsledky a jevy

Náhodný pokus je postup, jehož konkrétní výsledek předem neznáme, přestože známe množinu všech možných výsledků. Příkladem je hod hrací kostkou, hod mincí nebo rovnoměrný výběr bitového řetězce.

Definice 2.1. *Výběrový prostor* Ω je množina všech možných výsledků náhodného pokusu. *Náhodný jev* je libovolná podmnožina $A \subseteq \Omega$. Je nastane právě tehdy, když výsledek pokusu patří do množiny A .

Při hodu šestistěnnou kostkou máme

$$\Omega = \{1, 2, 3, 4, 5, 6\}.$$

Jev „padne sudé číslo“ odpovídá množině $A = \{2, 4, 6\}$ a jev „padne číslo větší než čtyři“ množině $B = \{5, 6\}$. Průnik $A \cap B = \{6\}$ nastane, když nastanou oba jevy současně. Sjednocení $A \cup B = \{2, 4, 5, 6\}$ nastane, když nastane alespoň jeden z nich.

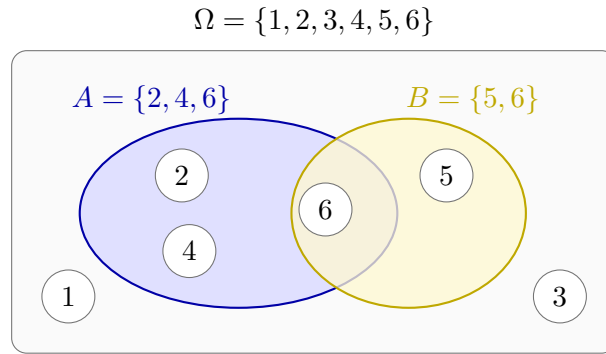


Figure 2: Výběrový prostor hodu kostkou a dva náhodné jevy.

Definice 2.2. Jsou-li všechny výsledky konečného výběrového prostoru Ω stejně pravděpodobné, definujeme pravděpodobnost jevu $A \subseteq \Omega$ vztahem

$$\Pr(A) = \frac{|A|}{|\Omega|}.$$

Pro sudé číslo na kostce tedy dostáváme

$$\Pr(A) = \frac{3}{6} = \frac{1}{2}.$$

Pravděpodobnost nemožného jevu je $\Pr(\emptyset) = 0$ a pravděpodobnost jistého jevu je $\Pr(\Omega) = 1$. Často ji zapisujeme také v procentech, například $\frac{1}{2} = 50\%$.

Jev, že A nenastane, značíme $A^c = \Omega \setminus A$. Platí

$$\Pr(A^c) = 1 - \Pr(A).$$

Tento jednoduchý vztah bude později velmi užitečný u narozeninového paradoxu: místo přímého počítání pravděpodobnosti, že vznikne alespoň jedna shoda, nejprve spočítáme pravděpodobnost, že nevznikne žádná.

Více kroků náhodného pokusu. Při dvou hodech mincí je potřeba rozlišovat pořadí výsledků:

$$\Omega = \{00, 01, 10, 11\},$$

kde například 01 znamená, že v prvním hodu padla nula a ve druhém jednička. Jsou-li oba hody nezávislé a obě hodnoty stejně pravděpodobné, má každý výsledek pravděpodobnost $\frac{1}{4}$.

Obecně existuje právě 2^n bitových řetězců délky n . Vybereme-li řetězec x rovnoměrně náhodně z $\{0, 1\}^n$, potom pro každý předem zvolený řetězec $a \in \{0, 1\}^n$ platí

$$\Pr_{x \in \{0,1\}^n}[x = a] = \frac{1}{2^n}.$$

Dolní index zde říká, odkud a jak náhodnou hodnotu vybíráme; výraz v hranatých závorkách popisuje jev, jehož pravděpodobnost měříme.

Příklad 2.3. Vyberme rovnoměrně náhodně $x \in \{0, 1\}^4$. Celkem existuje $2^4 = 16$ možností. Jev, že x začíná jedničkou, obsahuje osm řetězců, a proto má pravděpodobnost $\frac{8}{16} = \frac{1}{2}$. Jev $x = 1011$ obsahuje jediný výsledek a má pravděpodobnost $\frac{1}{16}$.

2.2 Podmíněná pravděpodobnost a nezávislost

Někdy už o výsledku něco víme a ptáme se, jak tato informace změní pravděpodobnost jiného jevu. Jestliže například víme, že na kostce padlo sudé číslo, zůstávají možné jen výsledky $\{2, 4, 6\}$.

Definice 2.4. Necht $A, B \subseteq \Omega$ a $\Pr(B) > 0$. *Podmíněnou pravděpodobnost* jevu A za podmínky B definujeme jako

$$\Pr(A | B) = \frac{\Pr(A \cap B)}{\Pr(B)}.$$

Pro jevy $A = \{2, 4, 6\}$ a $B = \{4, 5, 6\}$ při hodu kostkou je $A \cap B = \{4, 6\}$. Proto

$$\Pr(A | B) = \frac{\Pr(\{4, 6\})}{\Pr(\{4, 5, 6\})} = \frac{2/6}{3/6} = \frac{2}{3}.$$

Definice 2.5. Jevy A a B jsou *nezávislé*, jestliže informace o nastání jednoho nemění pravděpodobnost druhého, tedy

$$\Pr(A | B) = \Pr(A).$$

Je-li $\Pr(B) > 0$, je tato podmínka ekvivalentní rovnosti

$$\Pr(A \cap B) = \Pr(A) \Pr(B).$$

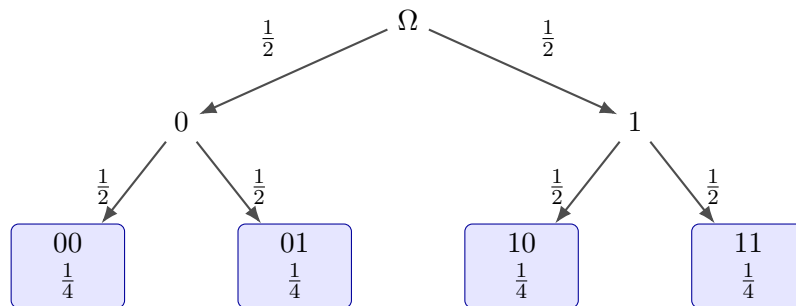


Figure 3: Dva nezávislé hody mincí a pravděpodobnosti výsledných řetězců.

V kryptografii je nezávislost klíče na zprávě zásadní. Pokud by volba klíče závisela na obsahu zprávy, mohl by způsob jeho generování sám prozrazovat informace. U jednorázové šifry budeme dokonce požadovat, aby znalost šifrovaného textu nezměnila pravděpodobnost žádné původní zprávy:

$$\Pr(M = m | C = c) = \Pr(M = m).$$

Náhodné veličiny.

Definice 2.6. *Náhodná veličina* přiřazuje každému výsledku náhodného pokusu nějakou hodnotu. Formálně jde o zobrazení $X : \Omega \rightarrow S$, kde S je množina možných hodnot veličiny.

Písmena M , K a C budeme používat pro náhodné veličiny představující postupně zprávu, klíč a šifrový text. Zápis $M = m$ označuje jev, že náhodná veličina M nabyla konkrétní hodnoty m . Rozdíl mezi M a m je tedy podobný rozdílu mezi proměnnou a její právě zvolenou hodnotou.

Pravděpodobnostní algoritmus může při běhu používat vlastní náhodné bity. Pokud píšeme pravděpodobnost jeho úspěchu, počítáme nejen s náhodným vstupem, ale také s těmito vnitřními volbami. Stejný algoritmus proto může na stejném vstupu v různých bězích postupovat odlišně.

3 Historické metody

Nejstarší šifry pracovaly přímo se znaky textu. Zprávu bylo možné šifrovat ručně bez počítače, jejich jednoduchost je však zároveň slabinou. Na historických metodách dobře uvidíme, proč velký počet klíčů sám o sobě ještě nezaručuje bezpečnost.

3.1 Substituční šifra s posunem

Mějme abecedu

$$\Sigma = \{a_0, a_1, \dots, a_{q-1}\}$$

a tajný posun k , kde $0 < k < q$. Každý znak nahradíme znakem posunutým o k míst:

$$a_i \mapsto a_{(i+k) \bmod q}.$$

Operace modulo q způsobí, že se po dosažení konce abecedy vrátíme na její začátek. Dešifrování používá opačný posun:

$$a_j \mapsto a_{(j-k) \bmod q}.$$

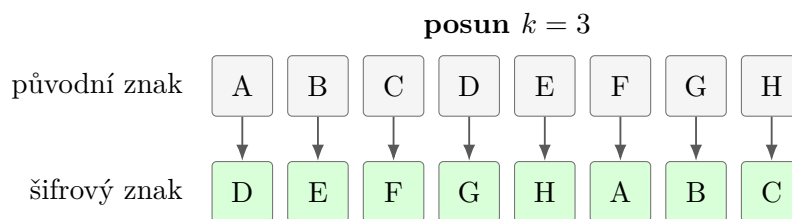


Figure 4: Substituční šifra s posunem $k = 3$ na zkrácené abecedě.

Pro standardní latinskou abecedu a $k = 3$ například dostaneme

$$\text{TAJNE} \mapsto \text{WDMQH}.$$

Jestliže záškodník nezná klíč, může jednoduše vyzkoušet všech $q - 1$ nenulových posunů. Pro $q = 26$ jde pouze o 25 možností. U dostatečně dlouhé zprávy navíc snadno pozná, který výsledek připomíná smysluplný text.

3.2 Obecná substituční šifra

Místo jednoho posunu můžeme zvolit libovolnou permutaci abecedy

$$\pi : \Sigma \rightarrow \Sigma.$$

Permutace je bijekce, takže každý znak má právě jeden zašifrovaný obraz a každý znak šifrové abecedy má právě jeden vzor. Šifrování nahradí každý znak a znakem $\pi(a)$, zatímco dešifrování používá inverzní permutaci π^{-1} .

Pro abecedu o q znacích existuje $q!$ různých permutací. U 26 písmen je to přibližně $4 \cdot 10^{26}$ klíčů, takže prosté vyzkoušení všech možností už není praktické. Obecná substituce však stále zachovává strukturu původního textu: stejné znaky se vždy mění na stejné znaky, opakovaná slova vytvářejí stejné vzory a četnosti písmen se pouze přejmenují.

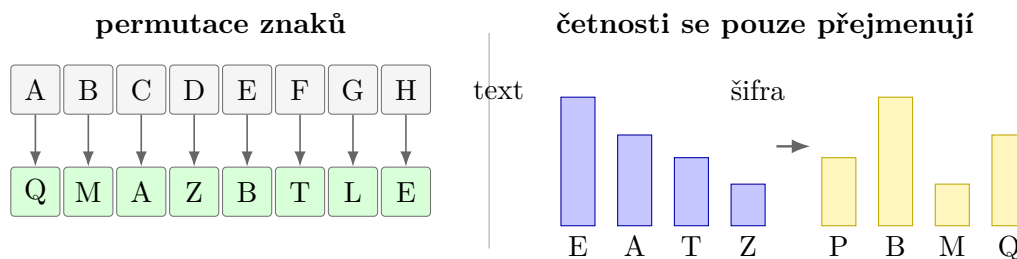


Figure 5: Obecná substituce a zachování četností znaků.

Uvažujme například klíč, jehož prvních několik přiřazení je

$$A \mapsto Q, \quad B \mapsto W, \quad C \mapsto E, \quad D \mapsto R, \quad E \mapsto T, \quad \dots$$

Pro permutaci použitou na obrázku 5 dostáváme

$$\text{TAJEMSTVI} \mapsto \text{ZQPTDLZCO}.$$

Velký počet klíčů neznamená automaticky bezpečnou šifru. Obecná substituce má obrovský prostor klíčů, ale frekvenční analýza využívá vlastnosti jazyka a nemusí jednotlivé klíče zkoušet jeden po druhém.

Frekvenční analýza. V delším českém textu se některá písmena objevují výrazně častěji než jiná. Jestliže se v šifrovaném textu mimořádně často vyskytuje znak Q, může odpovídat některému běžnému písmenu otevřeného textu. Záškodník dále sleduje dvojice a trojice znaků, délky slov nebo opakované vzory. Každý takový poznatek zmenšuje množinu možných permutací.

Historické metody nám proto ukazují dvě důležité zásady. Bezpečnost nelze odvozovat jen z toho, že klíčů je mnoho, a šifra by neměla zbytečně zachovávat statistickou strukturu otevřeného textu. Moderní šifry se snaží, aby bez znalosti klíče vypadal šifrovaný text jako náhodná data.

4 Symetrická kryptografie

V symetrické kryptografii používají Alice i Bob tentýž tajný klíč

$$k \in \{0, 1\}^n.$$

Alice vytvoří šifrový text $c = \mathbf{E}(m, k)$ a Bob obnoví zprávu výpočtem $m = \mathbf{D}(c, k)$. Klíč proto musí znát obě strany, ale nikdo další.

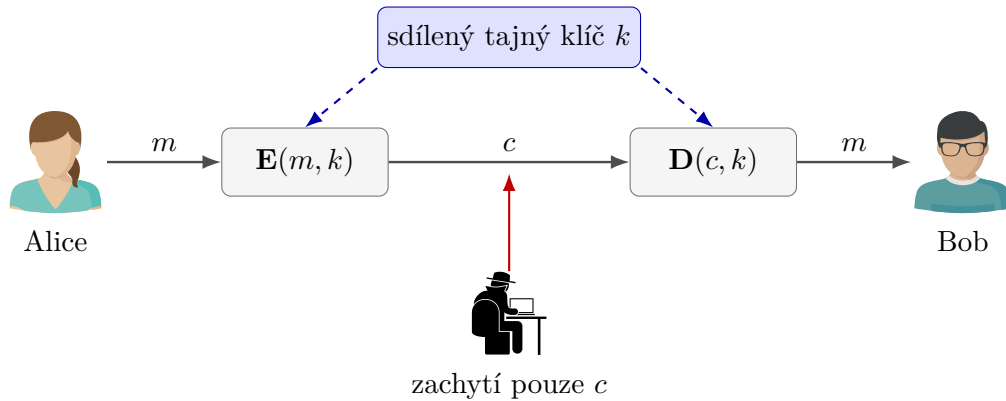


Figure 6: Základní schéma symetrického šifrování.

Symetrické algoritmy jsou obvykle velmi rychlé a hodí se pro šifrování velkého množství dat. Hlavní organizační problém spočívá v bezpečném předání klíče. Pokud Alice dokáže Bobovi tajně poslat dlouhý klíč, nabízí se otázka, proč stejným bezpečným kanálem neposlat přímo zprávu. Tento problém bude zvláště dobře vidět u jednorázové šifry.

4.1 Operace XOR

Jednorázová šifra používá operaci XOR, značenou symbolem $\oplus$. Pro jednotlivé bity platí

a	b	$a \oplus b$
0	0	0
0	1	1
1	0	1
1	1	0

Operaci na stejně dlouhých bitových řetězcích provádíme po jednotlivých pozicích. Důležitá je rovnost

$$(a \oplus b) \oplus b = a,$$

protože $b \oplus b = 0$ a $a \oplus 0 = a$.

4.2 Jednorázová šifra

Alice a Bob sdílejí rovnoměrně náhodný tajný klíč

$$k \in \{0, 1\}^n,$$

který je stejně dlouhý jako zpráva $m \in \{0, 1\}^n$. Alice šifruje

$$c = m \oplus k.$$

Bob použije stejný klíč a spočítá

$$c \oplus k = (m \oplus k) \oplus k = m.$$

Anglicky se tato metoda nazývá *one-time pad*; český název zdůrazňuje, že každý klíč smí být použit právě jednou.

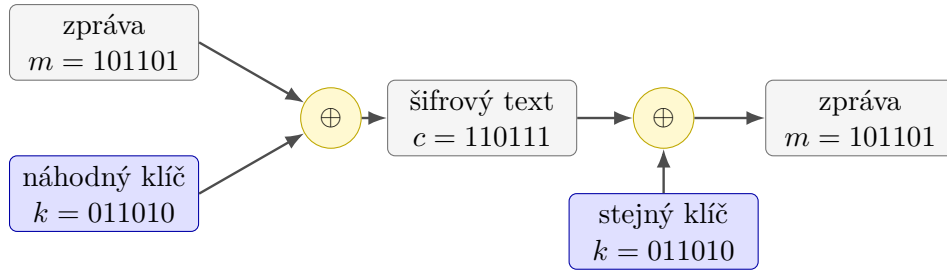


Figure 7: Šifrování a dešifrování jednorázovou šifrou.

Tvrzení 4.1. Necht K rovnoměrně náhodný na $\{0, 1\}^n$ a nezávislý na náhodné veličině M . Položme $C = M \oplus K$. Potom pro každé $m, c \in \{0, 1\}^n$ s $\Pr(M = m) > 0$ platí

$$\Pr(M = m \mid C = c) = \Pr(M = m).$$

Proof. Pro pevně zvolené hodnoty m a c existuje právě jeden klíč, který zprávu m převede na c , totiž

$$k = m \oplus c.$$

Protože je K rovnoměrně náhodný a nezávislý na M , dostáváme

$$\Pr(M = m \text{ a } C = c) = \Pr(M = m) \Pr(K = m \oplus c) = \frac{\Pr(M = m)}{2^n}.$$

Dále sečteme přes všechny možné zprávy m' :

$$\Pr(C = c) = \sum_{m' \in \{0, 1\}^n} \frac{\Pr(M = m')}{2^n} = \frac{1}{2^n}.$$

Z definice podmíněné pravděpodobnosti tedy plyne

$$\Pr(M = m \mid C = c) = \frac{\Pr(M = m \text{ a } C = c)}{\Pr(C = c)} = \Pr(M = m).$$

□

Zachycený šifrový text tedy nezmění pravděpodobnost žádné původní zprávy. Jednorázová šifra poskytuje *perfektní utajení*: její bezpečnost není založena na omezeném výpočetním výkonu záškodníka. Platí dokonce proti protivníkovi s libovolně výkonným počítačem.

Perfektní utajení platí pouze tehdy, když je klíč skutečně náhodný, stejně dlouhý jako zpráva, nezávislý na zprávě, bezpečně sdílený a použitý právě jednou.

4.3 Proč se klíč nesmí opakovat?

Zašifruje-li Alice dvě zprávy týměž klíčem k , dostaneme

$$c_1 = m_1 \oplus k, \quad c_2 = m_2 \oplus k.$$

Záškodník může oba zachycené texty spojit:

$$c_1 \oplus c_2 = (m_1 \oplus k) \oplus (m_2 \oplus k) = m_1 \oplus m_2.$$

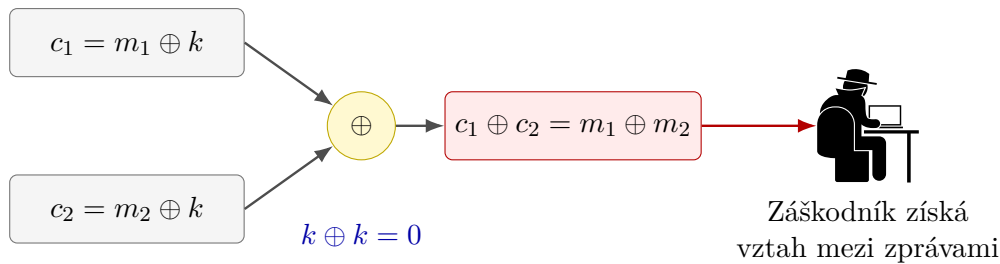


Figure 8: Opakované použití klíče odstraní z výrazu neznámé k .

Klíč se vyruší. Záškodník sice nemusí okamžitě znát obě zprávy, získá však informaci o jejich vzájemném vztahu. Pokud jednu z nich zná nebo správně odhadne, dopočítá druhou.

Nevýhodou jednorázové šifry je nutnost bezpečně sdílet náhodný klíč délky celé zprávy. V dalším výkladu uvidíme, jak lze perfektní bezpečnost nahradit bezpečností výpočetní a dlouhý náhodný klíč krátkým seedem.

5 Asymetrická kryptografie

Symetrické šifrování předpokládá, že Alice a Bob už sdílejí tajný klíč. Asymetrická kryptografie tento požadavek mění. Každý uživatel si vytvoří dvojici vzájemně souvisejících klíčů:

veřejný klíč k_V a soukromý klíč k_S .

Veřejný klíč smí znát kdokoli, soukromý klíč si jeho vlastník musí ponechat pouze pro sebe.

Chce-li Alice poslat zprávu Bobovi, získá Bobův veřejný klíč k_V a spočítá

$$c = \mathbf{E}(m, k_V).$$

Bob použije svůj soukromý klíč:

$$m = \mathbf{D}(c, k_S).$$

Korektnost požaduje

$$\mathbf{D}(\mathbf{E}(m, k_V), k_S) = m.$$

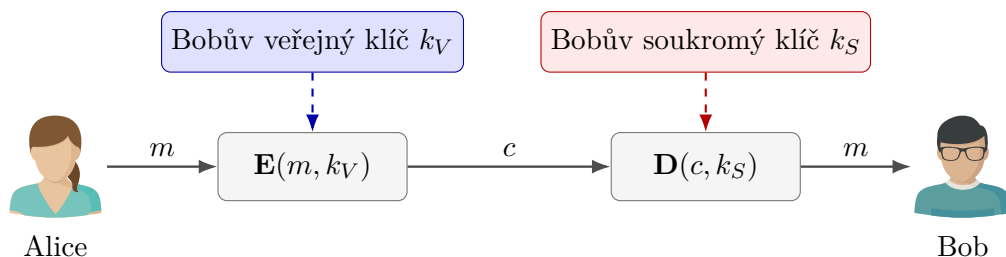


Figure 9: Šifrování Bobovým veřejným klíčem a dešifrování jeho soukromým klíčem.

Znalost veřejného klíče umožňuje zprávy pro Boba šifrovat, nesmí však umožnit je efektivně dešifrovat ani odvodit k_S . Asymetrie tedy není v tom, že by jedna operace byla matematicky nemožná, ale v rozdílu výpočetní obtížnosti: s tajnou informací je dešifrování snadné, bez ní má být prakticky neproveditelné.

Veřejný klíč musí být spolehlivě spojen s identitou svého vlastníka. Pokud záškodník podstrčí Alici svůj klíč a vydává jej za Bobův, může zprávu přechytit a teprve potom ji znovu zašifrovat skutečným Bobovým klíčem. K ověřování vazby mezi identitou a veřejným klíčem se používají certifikáty.

Asymetrické operace bývají výrazně pomalejší než symetrické. Reálné systémy proto často používají oba přístupy společně: asymetrická kryptografie bezpečně ustaví krátký dočasný klíč a samotná data se potom šifrují rychlou symetrickou šifrou. V této kapitole se soustředíme na princip; podrobnosti konkrétních síťových protokolů ponecháme stranou.

5.1 Výpočetní bezpečnost

U jednorázové šifry jsme dokázali, že šifrový text neposkytuje žádnou informaci ani neomezeně silnému záškodníkovi. U asymetrické kryptografie tak silný požadavek splnit nemůžeme: veřejný klíč i algoritmus šifrování jsou dostupné, takže záškodník může kandidátní zprávy sám šifrovat a porovnávat.

Bezpečnost proto formulujeme vůči algoritmům s omezenou dobou výpočtu. Pravděpodobnost úspěchu každého pravděpodobnostního polynomiálního útoku má být zanedbatelná. Tato myšlenka se naplno projeví v sekci o pseudonáhodných generátorech a jednosměrných funkcích.

Jednoduchý zápis $c = \mathbf{E}(m, k_V)$ nezobrazuje všechny praktické podrobnosti. Bezpečné asymetrické šifrování obvykle používá také náhodnost a předepsané kódování zprávy, aby stejné zprávy nevytvářely pokaždé stejný šifrový text.

6 Algoritmus RSA

RSA je nejznámější příklad asymetrického kryptografického systému. Jeho název vznikl z příjmení autorů Rona Rivesta, Adiho Shamira a Leonarda Adlemana. Základní operací je umocňování modulo součin dvou velkých prvočísel.

6.1 Vytvoření klíčů

Bob postupuje následovně:

1. zvolí dvě různá velká prvočísla p a q a spočítá

$$n = pq;$$

2. spočítá

$$\varphi(n) = (p - 1)(q - 1);$$

3. zvolí číslo s nesoudělné s $\varphi(n)$;

4. nalezne číslo v takové, že

$$vs \bmod \varphi(n) = 1;$$

5. zveřejní klíč $k_V = (v, n)$ a uchová v tajnosti klíč $k_S = (s, n)$.

Číslo v je multiplikativní inverze čísla s modulo $\varphi(n)$. Existuje právě proto, že $\gcd(s, \varphi(n)) = 1$, a lze je efektivně nalézt rozšířeným Eukleidovým algoritmem.

Číslo s je soukromý exponent, nikoli třetí prvočíslo. Musí být nesoudělné s $(p-1)(q-1)$, ale samo prvočíslem být nemusí.

6.2 Šifrování a dešifrování

Zprávu reprezentujeme číslem x splňujícím $0 \leq x < n$. Alice zašifruje

$$y = \mathbf{E}(x, k_V) = x^v \bmod n.$$

Bob dešifruje

$$\mathbf{D}(y, k_S) = y^s \bmod n.$$

Mocniny mohou být obrovské, v programu je však nepočítáme nejprve jako běžná celá čísla. Opakovaným umocňováním a průběžným výpočtem zbytku lze modulární mocninu určit v čase polynomiálním vzhledem k počtu bitů vstupu.

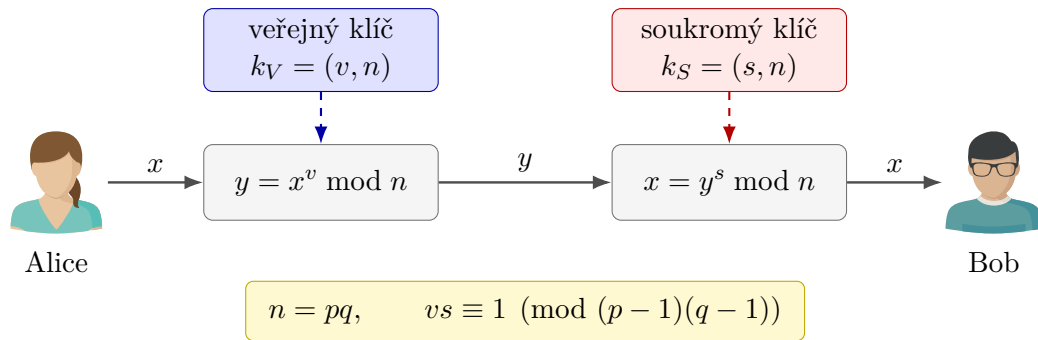


Figure 10: Schéma šifrování a dešifrování v RSA.

6.3 Malá Fermatova věta a čínská věta o zbytcích

Korektnost RSA odvodíme ze dvou vět elementární teorie čísel.

Věta 6.1 (Malá Fermatova věta). Nechť p je prvočíslo a $a \in \mathbb{Z}$ není dělitelné číslem p . Potom

$$a^{p-1} \bmod p = 1.$$

Ekvivalentně pro každé $a \in \mathbb{Z}$ platí $a^p \bmod p = a \bmod p$.

Věta 6.2 (Čínská věta o zbytcích). Nechť $m_1, \dots, m_k \in \mathbb{N}$ jsou po dvou nesoudělná. Pro libovolná $a_1, \dots, a_k \in \mathbb{Z}$ existuje řešení soustavy

$$x \equiv a_i \pmod{m_i}, \quad i = 1, \dots, k,$$

a všechna její řešení jsou navzájem shodná modulo $M = m_1 \cdots m_k$.

Důsledek 6.3. Jsou-li p a q různá prvočísla a platí

$$a \equiv b \pmod{p} \quad \text{a} \quad a \equiv b \pmod{q},$$

potom $a \equiv b \pmod{pq}$.

6.4 Korektnost RSA

Věta 6.4. Pro klíče vytvořené uvedeným postupem a každé celé číslo x splňující $0 \leq x < n$ platí

$$\mathbf{D}(\mathbf{E}(x, k_V), k_S) = x.$$

Proof. Z rovnosti $vs \bmod (p-1)(q-1) = 1$ plyne, že pro nějaké $t \in \mathbb{N}_0$ platí

$$vs = 1 + t(p-1)(q-1).$$

Nejprve ukážeme $x^{vs} \equiv x \pmod{p}$. Pokud p dělí x , jsou obě strany kongruence nulové modulo p . Pokud p číslo x nedělí, použijeme malou Fermatovu větu:

$$\begin{aligned} x^{vs} &= x^{1+t(p-1)(q-1)} \\ &= x \left(x^{p-1} \right)^{t(q-1)} \equiv x \cdot 1^{t(q-1)} \equiv x \pmod{p}. \end{aligned}$$

Stejným rozlišením případů dostaneme

$$x^{vs} \equiv x \pmod{q}.$$

Podle důsledku čínské věty o zbytcích tedy

$$x^{vs} \equiv x \pmod{pq}.$$

Protože $n = pq$ a $0 \leq x < n$, uzavíráme

$$\mathbf{D}(\mathbf{E}(x, k_V), k_S) = (x^v \bmod n)^s \bmod n = x^{vs} \bmod n = x.$$

□

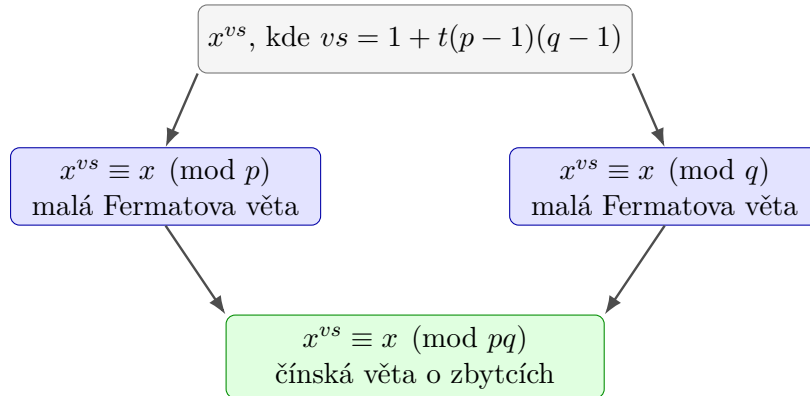


Figure 11: Myšlenka důkazu korektnosti RSA přes moduly p , q a pq .

Příklad s malými čísly.

Příklad 6.5. Zvolme $p = 5$, $q = 11$ a $s = 27$. Potom

$$n = 55, \quad \varphi(n) = (5-1)(11-1) = 40.$$

Protože $\gcd(27, 40) = 1$, můžeme nalézt v splňující

$$27v \bmod 40 = 1.$$

Vyhovuje $v = 3$, neboť $27 \cdot 3 = 81 \equiv 1 \pmod{40}$. Veřejný klíč je $k_V = (3, 55)$ a soukromý klíč $k_S = (27, 55)$.

Alice zašifruje $x = 7$:

$$y = 7^3 \bmod 55 = 343 \bmod 55 = 13.$$

Bob poté získá

$$13^{27} \bmod 55 = 7.$$

Malá čísla slouží pouze k ručnímu ověření postupu. V reálné kryptografii by tak malý modul záškodník rozložil okamžitě.

6.5 Bezpečnost a faktorizace

Veřejný klíč obsahuje $n = pq$, ale nikoli samotná prvočísla p a q . Pokud záškodník rozklad zjistí, spočítá $\varphi(n) = (p-1)(q-1)$ a z veřejného exponentu v odvodí soukromý exponent s . Efektivní faktorizace by tedy RSA prolomila.

Rozhodovací verzi problému můžeme zapsat takto:

FACTOR : pro daná $n, b \in \mathbb{N}$ rozhodni, zda existuje d takové, že $1 < d < b$ a $d \mid n$.

Jak jsme viděli v dřívějším výkladu tříd P a NP, kladnou odpověď stačí doložit dělitelem d . Ověření nerovnosti i rovnosti $n \bmod d = 0$ proběhne v polynomiálním čase, a proto FACTOR $\in$ NP.

Naivní zkoušení dělitelů je vzhledem k bitové délce $|\langle n \rangle_B|$ exponenciální. Jsou známy podstatně rychlejší klasické algoritmy, žádný obecný polynomiální klasický algoritmus pro faktorizaci však znám není.

Znalost efektivní faktorizace stačí k prolomení popsaného RSA. Není však dokázáno, že každý způsob, jak invertovat RSA, musí zároveň faktorizovat n . Bezpečnost RSA proto nelze jednoduše ztotožnit s větou „faktorizace je těžká“.

Uvedené „učebnicové RSA“ je deterministické a bez doplňujícího kódování není bezpečným praktickým šifrovacím schématem. Skutečné systémy používají standardizované náhodné vycpávání a oddělené postupy, například OAEP pro šifrování a PSS pro podpis.

7 Pseudonáhodné generátory a jednosměrné funkce

Jednorázová šifra ukázala, že dlouhý rovnoměrně náhodný klíč dokáže skrýt zprávu dokonale. Zároveň jsme narazili na její zásadní omezení: pro zprávu délky n potřebujeme předem sdílet n náhodných bitů. Moderní kryptografie se proto vzdává absolutního požadavku, aby šifrový text neobsahoval vůbec žádnou informaci, a nahrazuje jej požadavkem výpočetním. Pripouštíme, že informace může být v principu zjistitelná, ale žádný dostatečně rychlý algoritmus ji nemá umět prakticky využít.

Tento posun je zásadní. Výpočetní složitost zde není překážkou, kterou se snažíme odstranit, nýbrž prostředkem ochrany. Oprávněný uživatel zná krátký klíč nebo zvláštní tajnou informaci a výpočet

provede rychle. Záškodník stejnou informaci nemá a stojí před úlohou, pro kterou neznáme efektivní postup.

Technické definice v této sekci vyjadřují tři myšlenky:

1. oprávněný výpočet musí probíhat v polynomiálním čase;
2. záškodník smí používat libovolný pravděpodobnostní polynomiální algoritmus;
3. pravděpodobnost jeho úspěchu musí s rostoucí délkou vstupu klesat rychleji než převrácená hodnota libovolného polynomu.

Poslední bod zachytíme pojmem zanedbatelné funkce.

7.1 Zanedbatelná funkce

Definice 7.1. Funkce $\varepsilon : \mathbb{N} \rightarrow \mathbb{R}$ je *zanedbatelná*, jestliže pro každé $k \in \mathbb{N}$ existuje $n_k \in \mathbb{N}$ takové, že pro každé $n > n_k$ platí

$$|\varepsilon(n)| < \frac{1}{n^k}.$$

Zanedbatelná funkce tedy nakonec klesne pod $\frac{1}{n}$, pod $\frac{1}{n^2}$, pod $\frac{1}{n^{100}}$ i pod převrácenou hodnotu jakéhokoliv jiného pevně zvoleného polynomu. Typickým příkladem je 2^{-n} . Naopak funkce $\frac{1}{n^{10}}$ zanedbatelná není, protože pro $k = 11$ nikdy dlouhodobě neklesne pod $\frac{1}{n^{11}}$.

Číslo n zde hraje roli *bezpečnostního parametru*. Jeho zvětšením prodlužujeme klíče a zvyšujeme množství práce potřebné k útoku. Tvrzení o zanedbatelné pravděpodobnosti neznamená, že útok nemůže nikdy uspět; říká, že jeho šanci lze volbou dostatečně velkého parametru stlačit pod každou prakticky významnou inverzní polynomiální mez.

7.2 Pseudonáhodný generátor

Deterministický program nedokáže vytvořit skutečnou náhodnost z ničeho. Jakmile známe jeho vstup, je výstup pevně určen. Může však z krátkého náhodného vstupu vytvořit delší řetězec, který žádný efektivní algoritmus neumí od skutečně náhodného řetězce rozlišit.

Definice 7.2. Funkce

$$G : \{0, 1\}^n \rightarrow \{0, 1\}^{\ell(n)}$$

je *pseudonáhodný generátor* (PRG), jestliže splňuje následující podmínky:

1. G je vyčíslitelná deterministickým polynomiálním algoritmem;
2. $\ell(n) > n$ pro každé $n \in \mathbb{N}$;
3. pro každý pravděpodobnostní polynomiální algoritmus $\mathcal{A}$ existuje zanedbatelná funkce $\varepsilon(n)$ taková, že

$$\left| \Pr_{y \in \{0,1\}^{\ell(n)}}[\mathcal{A}(y) = 1] - \Pr_{y \in \{0,1\}^n}[\mathcal{A}(G(y)) = 1] \right| \leq \varepsilon(n).$$

Náhodný vstup $s \in \{0, 1\}^n$ nazýváme *seed*. Druhá podmínka zajišťuje *prodloužení* (*stretch*): výstup $G(s)$ obsahuje více bitů než seed. Nejdůležitější je třetí podmínka. Algoritmus $\mathcal{A}$ obdrží řetězec délky $\ell(n)$ a má rozhodnout, zda byl vybrán rovnoměrně náhodně, nebo zda vznikl jako $G(s)$ z náhodného seedu. Rozdíl jeho pravděpodobností přijetí v obou případech se nazývá rozlišovací výhoda.

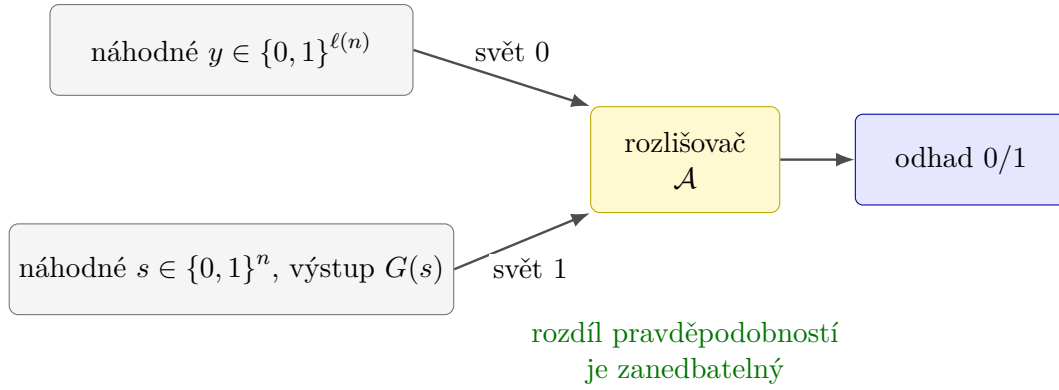


Figure 12: Experiment rozlišující skutečně náhodný řetězec od výstupu PRG.

Podmínka neříká, že jsou obě rozdělení stejná. Výstupů G může být nejvýše 2^n , zatímco všech řetězců délky $\ell(n)$ je $2^{\ell(n)}$. Většina dlouhých řetězců proto vůbec nemusí být výstupem generátoru. Rozdíl je informačně zjistitelný, avšak efektivní algoritmus nemá umět poznat, do které skupiny předložený řetězec patří.

Pseudonáhodnost není vlastnost jediného konkrétního řetězce. Je to vlastnost způsobu generování a říká, že celé rozdělení výstupů je pro efektivního pozorovatele nerozlišitelné od rovnoměrného rozdělení.

7.3 Žádné pseudonáhodné generátory při $P = NP$

Tvrzení 7.3. Jestliže $P = NP$, pak neexistuje žádný pseudonáhodný generátor.

Důkaz. Předpokládejme, že $P = NP$, a vezměme libovolnou polynomiálně vyčíslitelnou funkci

$$G : \{0,1\}^n \rightarrow \{0,1\}^{\ell(n)}, \quad \ell(n) > n.$$

Pro bezpečnostní parametr n označme množinu všech jejích možných výstupů

$$L_{G,n} = \{z \in \{0,1\}^{\ell(n)} \mid \text{existuje } s \in \{0,1\}^n \text{ takové, že } G(s) = z\}.$$

Ověřit, že $z \in L_{G,n}$, lze v nedeterministickém polynomiálním čase: certifikátem je seed $s \in \{0,1\}^n$ a ověřovatel pouze spočítá $G(s)$ a porovná výsledek se z . Při předpokladu $P = NP$ proto existuje deterministický polynomiální algoritmus $\mathcal{D}$, který rozhodne, zda $z \in L_{G,n}$:

$$\mathcal{D}(z) = 1 \iff z \in L_{G,n}.$$

Pokud rozlišovač dostane řetězec $G(s)$ vytvořený z náhodného seedu, přijme jej vždy:

$$\Pr_{s \in \{0,1\}^n} [\mathcal{D}(G(s)) = 1] = 1.$$

Množina $L_{G,n}$ obsahuje nejvýše 2^n řetězců, protože existuje pouze 2^n seedů. Pro rovnoměrně náhodný řetězec $z \in \{0,1\}^{\ell(n)}$ tedy platí

$$\Pr_{z \in \{0,1\}^{\ell(n)}} [\mathcal{D}(z) = 1] = \frac{|L_{G,n}|}{2^{\ell(n)}} \leq \frac{2^n}{2^{\ell(n)}} = 2^{n-\ell(n)}.$$

Celá rozlišovací výhoda algoritmu $\mathcal{D}$ je proto

$$\begin{aligned} & \left| \Pr_{z \in \{0,1\}^{\ell(n)}} [\mathcal{D}(z) = 1] - \Pr_{s \in \{0,1\}^n} [\mathcal{D}(G(s)) = 1] \right| \\ &= \left| \frac{|L_{G,n}|}{2^{\ell(n)}} - 1 \right| = 1 - \frac{|L_{G,n}|}{2^{\ell(n)}} \\ &\geq 1 - \frac{2^n}{2^{\ell(n)}} = 1 - 2^{n-\ell(n)} \geq 1 - \frac{1}{2} = \frac{1}{2}. \end{aligned}$$

Poslední nerovnost plyne z $\ell(n) > n$, a tedy $\ell(n) \geq n + 1$. Konstantní funkce $\frac{1}{2}$ není zanedbatelná, takže G nesplňuje podmínku nerozlišitelnosti z definice PRG. Protože jsme takto vyloučili libovolného kandidáta G , při rovnosti $P = NP$ žádný pseudonáhodný generátor neexistuje. $\square$

Jde o obměněnou implikaci tvrzení „existence PRG implikuje $P \neq NP$ “. Samotná nerovnost $P \neq NP$ však existenci pseudonáhodných generátorů nezaručuje.

7.4 Jednosměrná funkce

Pseudonáhodný generátor předpokládá existenci nějakého zdroje výpočetní obtížnosti. Tento zdroj formálně zachycují jednosměrné funkce: dopředný výpočet je snadný, ale návrat k libovolnému vzoru typického výstupu je pro efektivního záškodníka neproveditelný.

Definice 7.4. Zobrazení

$$f : \{0, 1\}^* \rightarrow \{0, 1\}^*$$

je *jednosměrná funkce* (OWF), jestliže:

1. hodnotu $f(x)$ lze pro každé $x \in \{0, 1\}^*$ spočítat v polynomiálním čase;
2. pro každý pravděpodobnostní polynomiální algoritmus $\mathcal{A}$ existuje zanedbatelná funkce $\varepsilon(n)$ taková, že pro každé $n \in \mathbb{N}$ platí

$$\Pr_{x \in \{0,1\}^n} [f(\mathcal{A}(f(x))) = f(x)] \leq \varepsilon(n).$$

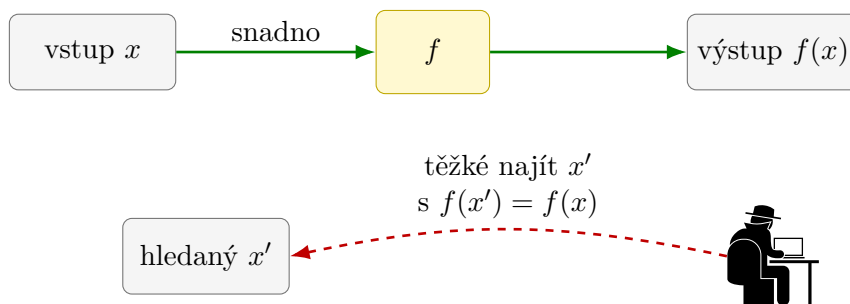


Figure 13: Dopředný výpočet jednosměrné funkce a obtížné hledání vzoru.

V experimentu nejprve rovnoměrně vybereme $x \in \{0, 1\}^n$, spočítáme $f(x)$ a výsledek předáme algoritmu $\mathcal{A}$. Algoritmus nemusí nalézt původní x . Uspěje i tehdy, když vrátí jiné x' se stejným obrazem, tedy $f(x') = f(x)$. Definice proto skutečně měří schopnost nalézt libovolný vzor, nikoli schopnost uhodnout jednu předem určenou reprezentaci.

První podmínka je nutná pro oprávněné použití: funkci musíme umět rychle vyčíslit. Druhá podmínka nepožaduje obtížnost pouze na jednom pečlivě vybraném vstupu. Vstup x je náhodný a úspěch musí být zanedbatelný v průměru nad všemi takovými volbami i nad náhodnými kroky algoritmu $\mathcal{A}$. To je pro kryptografii podstatné. Funkce, která je těžká pouze na několika výjimečných vstupech, by nechránila běžně generované klíče.

Nejhorší případ známý z teorie složitosti pro kryptografii nestačí. Systém není bezpečný, pokud záškodník snadno prolomí třeba polovinu náhodně generovaných klíčů, i kdyby zbývající případy byly mimořádně obtížné.

Kandidáti a padací vrátka. Není dokázáno, že jednosměrné funkce existují. Známe však přirozené kandidáty:

- násobení dvou velkých prvočísel $f(p, q) = pq$; dopředný výpočet je snadný, zatímco zpětný rozklad součinu je považován za obtížný;
- varianty problému součtu podmnožiny;
- modulární umocňování, jehož inverze souvisí s problémem diskretního logaritmu.

U žádného z těchto kandidátů dnes neumíme bez dalších předpokladů dokázat požadovanou průměrnou obtížnost.

Pro asymetrickou kryptografii potřebujeme ještě něco navíc. Vlastník soukromého klíče musí mít tajnou informaci, která obtížnou inverzi zjednoduší. Takové informaci se říká *padací vrátka* (*trapdoor*). RSA lze didakticky chápat právě tímto způsobem: bez znalosti p a q vypadá inverze modulárního umocňování obtížně, zatímco vlastník soukromého exponentu s dešifruje efektivně.

7.5 Souvislost OWF, PRG a šifrování

Mezi oběma technickými pojmy platí hluboká věta:

Věta 7.5. Jednosměrné funkce existují právě tehdy, když existují pseudonáhodné generátory.

Úplný důkaz je výrazně nad rámec tohoto textu. Důležitý je jeho význam: schopnost provést snadný dopředný, ale průměrně obtížně invertovatelný výpočet stačí k postupné konstrukci bitů, které vypadají náhodně. Pseudonáhodné generátory tedy nejsou nezávislý zázrak; jejich existence je založena na témže druhu výpočetní obtížnosti jako velká část moderní kryptografie.

V praxi můžeme krátký tajný seed $s \in \{0, 1\}^n$ rozvinout na dlouhý řetězec $G(s) \in \{0, 1\}^{\ell(n)}$ a šifrovat podobně jako jednorázovou šifrou:

$$c = m \oplus G(s).$$

Bob se stejným seedem vygeneruje tentýž proud a spočítá

$$c \oplus G(s) = m.$$

Oproti jednorázové šifře už výstup $G(s)$ není skutečně náhodný, a proto nezískáváme perfektní utajení. Pokud jej však žádný efektivní algoritmus nerozliší od náhodného řetězce, získáváme bezpečnost výpočetní. Tato výměna dramaticky zmenší množství tajných dat, která musí Alice a Bob sdílet.

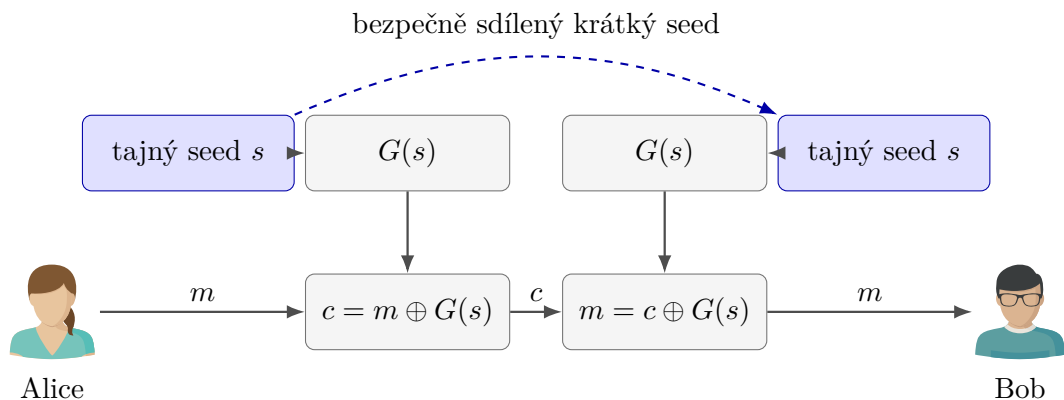


Figure 14: Použití krátkého seedu k vytvoření dlouhého šifrovacího proudu.

Samotný deterministický proud $G(s)$ se nesmí bez dalšího opakovat pro více zpráv. Praktické proudové šifry kombinují klíč s jedinečnou veřejnou hodnotou, například nonce, aby pro každé šifrování vznikl jiný proud bitů.

8 Hešovací funkce

Šifrování převádí zprávu na šifrový text, který má oprávněný příjemce znovu dešifrovat. Hešovací funkce má jiný účel. Z libovolně dlouhých dat vytvoří krátký otisk pevné délky. Z otisku se původní data běžně neobnovují; používá se především k rychlému porovnání dat, kontrole integrity a jako součást dalších kryptografických konstrukcí.

Definice 8.1. *Hešovací funkce* je zobrazení

$$\mathcal{H}_n : \{0, 1\}^* \rightarrow \{0, 1\}^n.$$

Pro $x \in \{0, 1\}^*$ nazýváme hodnotu $\mathcal{H}_n(x)$ *hash* nebo *otisk* řetězce x .

Index n připomíná délku výstupu. Ať je vstupem krátké heslo, rozsáhlý dokument nebo obraz disku, výsledkem je vždy právě n bitů. Kryptografická hešovací funkce má být rychle vyčíslitelná a malá změna vstupu má typicky změnit velkou část výstupních bitů. Tato citlivost se často označuje jako lavinový efekt.

Lavinový efekt je užitečná konstrukční vlastnost, sám o sobě však nedokazuje bezpečnost. Funkce může na pohled měnit výstup chaoticky a přesto umožňovat efektivní hledání vzorů nebo kolizí.

8.1 Kolize jsou nevyhnutelné

Množina $\{0, 1\}^*$ je nekonečná, zatímco množina $\{0, 1\}^n$ obsahuje pouze 2^n prvků. Z Dirichletova principu proto plyne, že existují různá data $x \neq y$ se stejným otiskem:

$$\mathcal{H}_n(x) = \mathcal{H}_n(y).$$

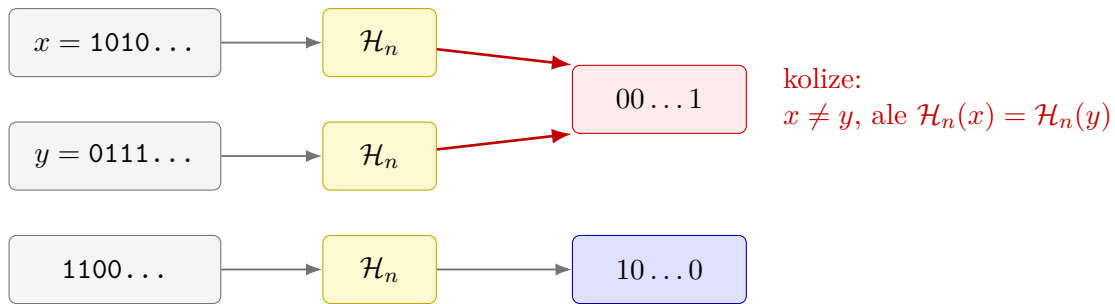


Figure 15: Více vstupů se musí zobrazit na tentýž n -bitový otisk.

Takovou dvojici nazýváme *kolize*.

Kolizím tedy nelze zabránit matematicky. Bezpečnostní požadavek musí znít jinak: kolize existují, ale žádný efektivní algoritmus je nemá umět nalézt s nezanedbatelnou pravděpodobností.

8.2 Narozeninový paradox

Nejprve uvažujme skupinu k lidí a zjednodušující předpoklad, že každý z 365 dnů narození je stejně pravděpodobný a výběry jsou nezávislé. Pravděpodobnost, že všech k narozenin připadne na různé dny, je

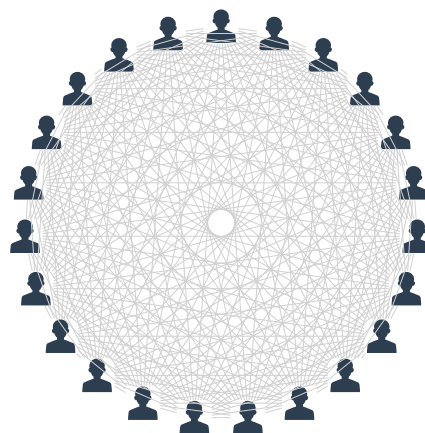
$$\Pr(C = 0) = \frac{365}{365} \cdot \frac{364}{365} \cdots \frac{365 - k + 1}{365}.$$

Pravděpodobnost alespoň jedné shody proto získáme doplňkem:

$$\Pr(C > 0) = 1 - \prod_{i=0}^{k-1} \frac{365 - i}{365}.$$

Už pro $k = 23$ vychází přibližně 50,7 %. Důvodem je počet dvojic: ve skupině 23 lidí jich existuje

$$\binom{23}{2} = 253.$$



23 lidí vytváří $\binom{23}{2} = 253$ dvojic

Figure 16: U 23 lidí porovnáváme 253 různých dvojic.

Stejný výpočet použijeme na r -bitové otisky. Předpokládejme pro hrubý odhad, že se otisky chovají jako nezávislé rovnoměrně náhodné hodnoty z množiny o 2^r prvcích. Pro k různých zpráv je pravděpodobnost alespoň jedné kolize

$$\begin{aligned}\Pr(C > 0) &= 1 - \prod_{j=1}^{k-1} \left(1 - \frac{j}{2^r}\right) \\ &\approx 1 - \exp\left(-\frac{k(k-1)}{2^{r+1}}\right).\end{aligned}$$

Použili jsme přibližnou rovnost $1 - x \approx e^{-x}$ pro malé x a součet

$$\sum_{j=1}^{k-1} j = \frac{k(k-1)}{2}.$$

Pro $k \approx 2^{r/2}$ dostáváme

$$\Pr(C > 0) \approx 1 - e^{-1/2} \approx 0,3935.$$

K pravděpodobnosti blízké jedné polovině stačí jen o konstantní faktor více pokusů. Hrubou silou lze proto kolizi r -bitové hešovací funkce hledat řádově v čase $2^{r/2}$, nikoli až 2^r .

Výstup délky r bitů poskytuje proti obecnému hledání kolizí přibližně jen $r/2$ bitů bezpečnosti. Právě proto kryptografické hešovací funkce používají poměrně dlouhé otisky.

8.3 Bezpečnostní vlastnosti

Uvažujme rodinu hešovacích funkcí

$$\mathcal{H}_n : \{0, 1\}^* \rightarrow \{0, 1\}^n$$

a pravděpodobnostní polynomiální algoritmus $\mathcal{A}$, který představuje záškodníka.

Odolnost vůči nalezení vzoru. Záškodník dostane otisk náhodně zvoleného vstupu a snaží se najít libovolná data se stejným otiskem. Formálně požadujeme, aby pro nějakou zanedbatelnou funkci ε platilo

$$\Pr_{x \in \{0,1\}^n} [\mathcal{H}_n(\mathcal{A}(\mathcal{H}_n(x))) = \mathcal{H}_n(x)] \leq \varepsilon(n).$$

Algoritmus nemusí vrátit původní x ; úspěchem je jakýkoliv vzor daného otisku.

Odolnost vůči kolizím. Záškodník se snaží vytvořit libovolné dva různé vstupy se stejným otiskem. Požadujeme

$$\Pr \left[\begin{array}{l} \mathcal{A} \text{ vypíše } (x, y), \\ x \neq y \text{ a } \mathcal{H}_n(x) = \mathcal{H}_n(y) \end{array} \right] \leq \varepsilon(n),$$

kde pravděpodobnost počítáme přes náhodné volby algoritmu $\mathcal{A}$.

Tyto požadavky nelze zaměňovat s pouhou malou pravděpodobností, že se srazí dva náhodně vybrané vstupy. Odolnost vůči kolizím se ptá na aktivního záškodníka, který vstupy volí záměrně a může využít strukturu algoritmu $\mathcal{H}_n$.

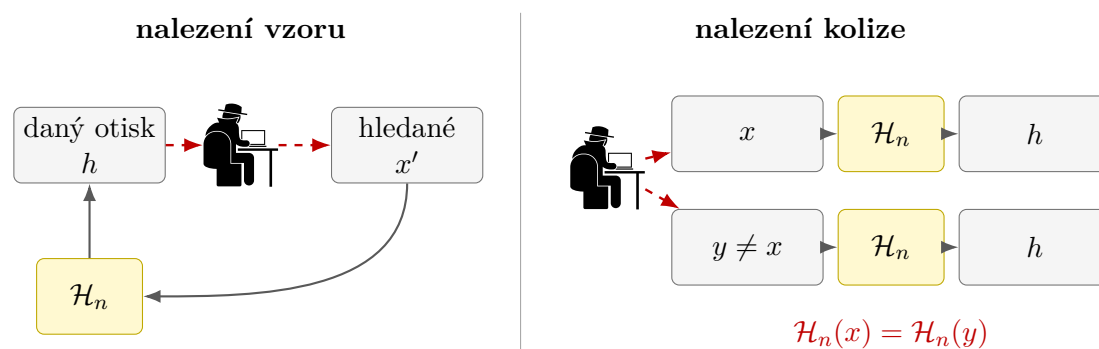


Figure 17: Rozdíl mezi hledáním vzoru daného otisku a hledáním libovolné kolize.

Vztah k jednosměrným funkcím. Odolnost vůči nalezení vzoru připomíná definici jednosměrné funkce: dopředný výpočet $\mathcal{H}_n(x)$ je rychlý, ale z výstupu má být těžké získat libovolný odpovídající vstup. Bezpečná hešovací funkce navíc požaduje obtížné hledání kolizí. Ne každá jednosměrná funkce proto musí být vhodnou hešovací funkcí a ne každá rychlá funkce s krátkým výstupem splňuje kryptografické požadavky.

Hešovací funkce není šifrování bez klíče. Šifrování je navrženo tak, aby oprávněný příjemce mohl data obnovit. U hešování se žádná dešifrovací operace nepředpokládá a ztráta informace je záměrná.

9 Secure Hash Algorithm

Zkratka SHA pochází z anglického názvu *Secure Hash Algorithm*. Neoznačuje jedinou funkci, ale několik rodin standardizovaných kryptografických hešovacích funkcí.

- **SHA-0** byl zveřejněn v roce 1993 jako původní Secure Hash Standard.
- **SHA-1** jej v roce 1995 nahradil. Později byly nalezeny praktické kolize, a proto se pro nové bezpečnostní použití nepoužívá.
- **SHA-2** je rodina funkcí zahrnující například SHA-224, SHA-256, SHA-384 a SHA-512.
- **SHA-3** byl standardizován v roce 2015 a vychází z algoritmu Keccak. Má odlišnou vnitřní konstrukci než SHA-2.

Pro nové aplikace se podle konkrétního standardu a protokolu používají zejména funkce z rodin SHA-2 a SHA-3. Skutečnost, že obě rodiny zůstávají v použití, není rozpor: mají jinou konstrukci a mohou představovat nezávislé možnosti pro různé systémy.

9.1 Sponge function

SHA-3 používá konstrukci nazývanou *sponge function*, doslova „houbová funkce“. Název připomíná dvě fáze: v první funkci postupně „nasává“ bloky zprávy a ve druhé ze svého vnitřního stavu „vymačkává“ výstupní bity.

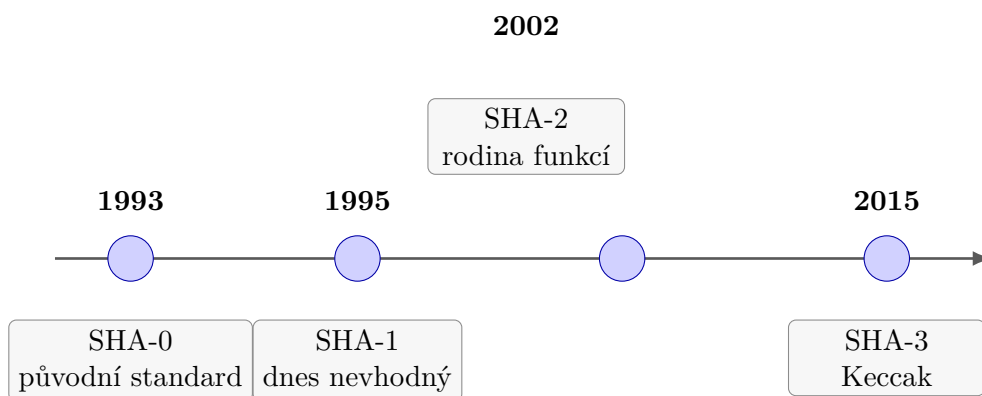


Figure 18: Zjednodušená časová osa rodin SHA.

Vnitřní stav má délku

$$b = r + c.$$

Prvních r bitů tvoří část *rate*, která komunikuje se vstupem a výstupem. Zbývajících c bitů tvoří *capacity*; nejsou přímo vydávány jako výstup a významně ovlivňují bezpečnost konstrukce. Celý stav zpracovává veřejná kryptografická permutace

$$f : \{0, 1\}^b \rightarrow \{0, 1\}^b.$$

Fáze absorbing. Zprávu m nejprve jednoznačně doplníme předepsaným paddingem a rozdělíme na r -bitové bloky

$$p_1, p_2, \dots, p_t.$$

Začneme nulovým stavem $S_0 = 0^b$. Každý blok prodloužíme o c nulových bitů, zkombinujeme s předchozím stavem operací XOR a na výsledek použijeme permutaci f :

$$S_i = f(S_{i-1} \oplus (p_i 0^c)), \quad i = 1, \dots, t.$$

Řetězec $p_i 0^c$ má délku $r + c = b$, takže jej lze XORovat s celým stavem. Vstupní blok ovlivní po aplikaci permutace obě části stavu.

Fáze squeezing. Po zpracování všech bloků čteme bity z části rate výsledného stavu S_t . Potřebujeme-li více než r výstupních bitů, znovu použijeme permutaci f a z nového stavu odebereme další část rate. Proces opakujeme, dokud nemáme požadovanou délku otisku.

Permutace f není tajný klíč. Je veřejnou a pevně určenou součástí algoritmu. Bezpečnost má vycházet z její konstrukce a ze způsobu, jakým se celý vnitřní stav používá, nikoli z utajení postupu.

9.2 SHA-3 jako konkrétní sponge function

SHA-3 používá stav délky

$$b = 1600.$$

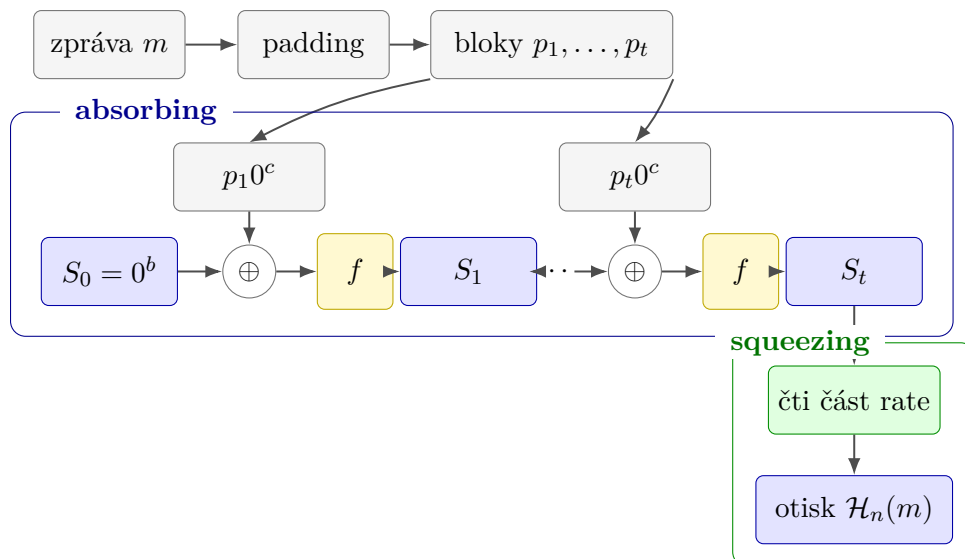


Figure 19: Fáze absorbing a squeezing konstrukce sponge function.

Pro variantu SHA3- d s d -bitovým otiskem se volí

$$c = 2d, \quad r = 1600 - c.$$

Například SHA3-256 má

$$d = 256, \quad c = 512, \quad r = 1088.$$

Další běžné varianty jsou SHA3-224, SHA3-384 a SHA3-512. Liší se délkou otisku, a tím i rozdělením stavu na rate a capacity.

SHA-3 je navržen jako bezpečná kryptografická hešovací funkce, formální důkaz, že konkrétní SHA3- d splňuje definici jednosměrné funkce z předchozího výkladu, však znám není. To není u praktických kryptografických funkcí neobvyklé: bezpečnost se opírá o rozsáhlou veřejnou analýzu, známé útoky a přesně popsané bezpečnostní předpoklady.

Délka otisku a vnitřní konstrukce řeší různé otázky. Delší otisk zvětšuje prostor možných hodnot, ale bezpečnost stále závisí také na tom, zda ve funkci neexistuje strukturální zkratka rychlejší než obecný útok hrubou silou.

10 Elektronický podpis

Šifrování chrání důvěrnost, ale samo o sobě nemusí prokazovat původ zprávy. Bob může obdržet šifrový text, který dokáže otevřít svým soukromým klíčem, a přesto neví, zda jej skutečně vytvořila Alice. Elektronický podpis má propojit konkrétní obsah zprávy se soukromým klíčem podepisující osoby.

Proč nestačí připsat jméno? Představme si, že Alice pošle

$$\mathbf{E}(x, k_V^B)$$

a vedle šifrovaného textu pouze připojí značku „Alice“. Záškodník P může zprávu zachytit, značku nahradit a odeslat dál

$$\mathbf{E}(x, k_V^B); P.$$

Bob zprávu správně dešifruje, ale mylně ji přisoudí záškodníkovi. Obyčejný textový popis není se zprávou kryptograficky svázán.

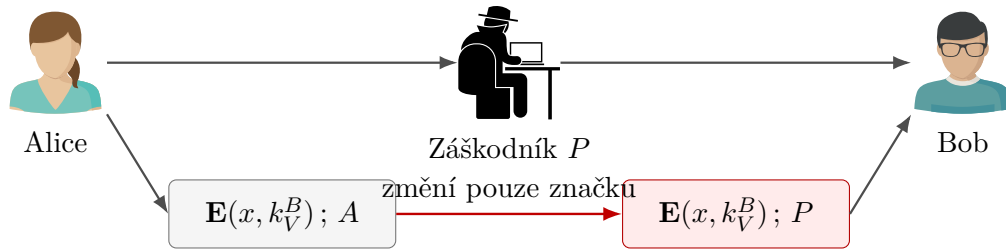


Figure 20: Záškodník změnil nepodepsanou značku odesílatele.

Podpis musí záviset na obsahu zprávy i na tajné informaci podepisující osoby. Změní-li se zpráva, starý podpis už nesmí projít ověřením. Zároveň jej nesmí umět vytvořit nikdo, kdo nezná Alicin soukromý klíč k_S^A .

Komutativita klíčů jako didaktická představa. U učebnicového RSA platí nejen

$$D(E(x, k_V), k_S) = x,$$

ale díky násobení exponentů také obrácené pořadí

$$E(D(x, k_S), k_V) = x.$$

To nabízí jednoduchou představu podpisu. Alice použije na hodnotu x svůj soukromý klíč:

$$\sigma = D(x, k_S^A).$$

Bob použije Alicin veřejný klíč a zkontroluje

$$E(\sigma, k_V^A) \stackrel{?}{=} x.$$

Protože k_V^A je veřejný, podpis může ověřit kdokoli. Vytvořit odpovídající σ však má umět pouze Alice se svým k_S^A .

Obecný elektronický podpis není „dešifrování soukromým klíčem“. Jde pouze o názornou intuici založenou na algebraické vlastnosti učebnicového RSA. Praktické podpisové schéma má samostatný algoritmus podepsání, algoritmus ověření a bezpečné kódování dat.

10.1 Podpis otisku

Podepisovat přímo dlouhou zprávu by bylo pomalé a nepraktické. Proto použijeme hešovací funkci

$$\mathcal{H}_n : \{0, 1\}^* \rightarrow \{0, 1\}^n.$$

Alice nejprve spočítá krátký otisk $h = \mathcal{H}_n(m)$ a podepíše až jej:

$$\sigma = D(\mathcal{H}_n(m), k_S^A).$$

Bob z přijaté zprávy sám vypočítá $\mathcal{H}_n(m)$ a ověří

$$E(\sigma, k_V^A) \stackrel{?}{=} \mathcal{H}_n(m).$$

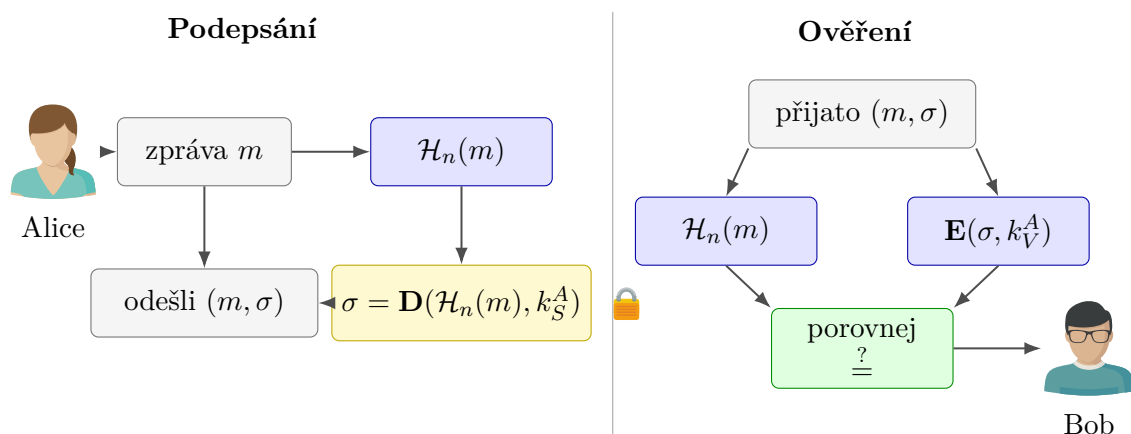


Figure 21: Podepsání otisku zprávy a následné ověření podpisu.

Pokud někdo zprávu pozmění, její nový otisk se s hodnotou získanou z podpisu neshoduje.

Odolnost hešovací funkce vůči kolizím je zde zásadní. Pokud by záškodník dokázal najít dvě různé zprávy $m \neq m'$ se stejným otiskem, mohl by se pokusit získat podpis neškodné zprávy m a později jej připojit k nebezpečné zprávě m' .

10.2 Certifikát a ověření veřejného klíče

Bob potřebuje vědět, že klíč k_V^A opravdu patří Alici. *Digitální certifikát* spojuje identitu držitele s jeho veřejným klíčem a toto spojení potvrzuje další důvěryhodná autorita svým podpisem. Ověření certifikátu proto není pouhým přečtením jména v souboru; příjemce kontroluje podpis vydavatele a navazující řetězec důvěry.



Figure 22: Certifikát kryptograficky spojuje identitu Alice s jejím veřejným klíčem.

Certifikát neukládá Alicin soukromý klíč. Ten musí zůstat pod její kontrolou. Obsahuje zejména identifikační údaje, veřejný klíč, dobu platnosti a podpis vydavatele. Při ověřování je dále potřeba zkontrolovat, zda certifikát nevypršel nebo nebyl zneplatněn.

Platný podpis dokládá, že podpis vytvořil někdo s přístupem k odpovídajícímu soukromému klíči a že se podepsaná data nezměnila. Pokud byl soukromý klíč odcizen nebo s ním bylo neoprávněně nakládáno, samotná matematická kontrola tuto okolnost neodhalí.

10.3 Elektronický podpis v právním rámci

Technický pojem elektronického podpisu je potřeba odlišovat od jeho právních kategorií. V Evropské unii oblast upravuje zejména nařízení eIDAS č. 910/2014 v aktuálním konsolidovaném znění. Rozlišuje elektronický podpis, zaručený elektronický podpis a kvalifikovaný elektronický podpis. Kvalifikovanému elektronickému podpisu přiznává právní účinek rovnocenný vlastnoručnímu podpisu; jinému elektronickému podpisu nelze právní účinek a přípustnost jako důkazu odmítnout jen proto, že má elektronickou podobu nebo nesplňuje požadavky kvalifikovaného podpisu.

V České republice na evropskou úpravu navazuje zejména zákon č. 297/2016 Sb., o službách vytvářejících důvěru pro elektronické transakce. Informace o kvalifikovaných poskytovatelích a službách zveřejňuje Digitální a informační agentura.

Právní předpisy a seznamy poskytovatelů se mohou měnit. Při skutečném právním jednání je proto potřeba ověřit aktuální znění předpisů, požadovaný druh podpisu a platnost použitého certifikátu.

Co podpis zajišťuje a co nikoli. Správně navržený a ověřený elektronický podpis pomáhá prokázat integritu dat a vazbu na soukromý klíč podepisující osoby. Nezajišťuje však důvěrnost: podepsaná zpráva může zůstat veřejně čitelná. Potřebujeme-li obsah zároveň utajit, kombinujeme podpis se šifrováním.

Pořadí a konkrétní způsob této kombinace musí určovat bezpečný protokol. Nestačí libovolně spojit dvě samostatně bezpečné operace. Moderní knihovny proto nabízejí hotová standardizovaná podpisová a šifrovací schémata; vytvářet vlastní kryptografický protokol bez odborné analýzy je velmi riskantní.

Při ověřování podpisu kontrolujeme současně tři vazby: otisk odpovídá přijatým datům, podpis odpovídá veřejnému klíči a certifikát důvěryhodně spojuje tento veřejný klíč s identitou podepisující osoby.